AZINE

Architecture

How azine is built, and why

OCTOBER 31, 2026
AZINE.IO · MAIN @ 5036B91

1. What azine is
2. The system at a glance
3. Hosting and deployment
4. Phones: the home-screen app
5. Back end: the server
6. Content: catalogs and enrichment
7. Finding things: filters, groups and search
8. Front end: the four letters
9. Operations
10. Decision log

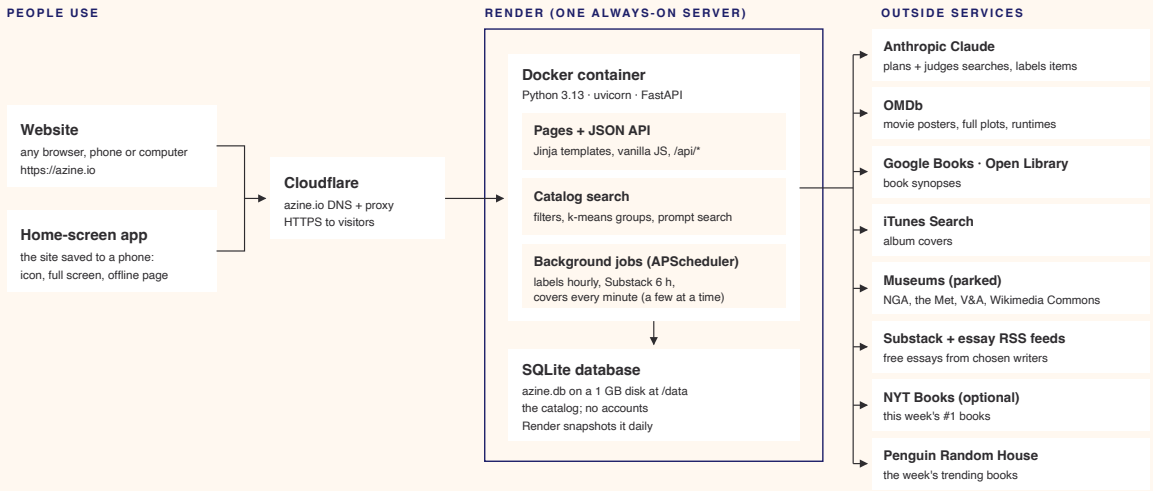
01 What azine is

A catalog of essays, movies, albums and books (E-M-A-B), the same for everyone. Every entry is a real, curated work, mostly pre-2021 (plus festival films from 2022 on); nothing is AI-written. No accounts, personal picks or votes. Find things by category, by year made, by country (not essays), and by describing what you want.

WHY

The promise is "the best of the internet without the filler", with nothing between a visitor and the work. Until Oct 27, 2026 azine was a personal board built from a taste answer and votes. Claude works only behind the scenes: it reads search prompts, judges fit and labels items. It never writes what you read.

02 The system at a glance



One Python server on Render with SQLite on an attached disk. Cloudflare serves azine.io; outside services are called only to read search prompts, label items, and fetch covers and synopses.

A search, end to end

1 Cloudflare resolves `azine.io`; Render serves it over HTTPS.

2 Browsing (`GET /api/catalog`) is a plain database query: no Claude.

3 A prompt (`POST /api/search`) is checked in plain code for prompt injection, then Claude Sonnet turns it into a search plan inside the filters already set.

4 The server lines up every work in the filters, most similar first; Claude Haiku scores each, and every 75+ comes back, best first (~25 s for 1,700 films).

5 Anything shown without a cover is fetched in the background within a minute.

03 Hosting and deployment

PIECE	SETUP
Host	Render web service, Docker, Oregon, 0.5 CPU / 512 MB (~$7/month), always on
Image	`uv` + Python 3.13 slim; runs `uvicorn azine.main:app --proxy-headers` on `$PORT`
Disk	1 GB at `/data`, `AZINE_DB_PATH=/data/azine.db`; keeps covers, synopses and labels across deploys
Config	`render.yaml`: disk, `/health`, env vars. Secrets (`ANTHROPIC_API_KEY`, `OMDB_API_KEY`, `GOOGLE_BOOKS_API_KEY`, `NYT_API_KEY`) live in the dashboard, never in git
Domain	azine.io on Cloudflare: `A @ → 216.24.57.1`, `CNAME www → azine.onrender.com`; Render issues the certificate
Deploys	Every push to `main`; ~1 minute of downtime (the disk forces stop-then-start); migrations run on start
Security	No accounts, cookies or passwords. Proxy headers identify visitors for the search rate limit

WHY ONE SERVER, SQLITE AND A DISK

One database file, a few background jobs and a few hundred requests a day need nothing more: no database server, job queue or cluster. The trade-off is deliberate: exactly one instance, and the server refuses to start without the disk. To grow, use a bigger instance; Postgres and a separate worker only if azine ever needs two servers.

WHY RENDER

Vercel, Netlify, Cloudflare Pages and Heroku lack a persistent disk or an always-on process. Fly.io fits but is driven from a CLI; Render has a dashboard and deploys from GitHub. Render's free tier sleeps and has no disk. Docker keeps it portable to any host with a disk and HTTPS.

04 Phones: the home-screen app

The website, saved to a phone's home screen, gets an icon, full screen and an offline page (`manifest.webmanifest`, icons, `/sw.js`). There is no App Store app: the home-screen app updates with every deploy, and Apple rejects apps that only wrap a website (guideline 4.2).

The service worker is **network-first**: pages and the API are never served from cache; `/static/` is cached only as an offline fallback. The stylesheet URL carries its modification time (`style.css?v=…`), so a new page is never paired with an old stylesheet, which once left the site unstyled.

05 Back end: the server

Code (`src/azine/`)

MODULE	JOB
main.py	The FastAPI app; on start runs migrations, seeds the catalog, starts the scheduler
db.py · models.py · migrations/	SQLAlchemy over SQLite; Alembic migrations applied on every start
routers/	`catalog.py` (the API), `pages.py` (pages, robots, sitemap); `art.py` parked
catalog/	Filters and sorting, embeddings, groups, search, `prompt_guard.py`, enrichment (`enrich.py`, `details.py`)
claude.py · labeling.py	The one structured-output call to Claude; subject, tone and country labels
content_sources/ · data/	Catalog loaders and live sources; curated lists and vocabularies
catalog_builder.py · scheduler.py · seed.py	Offline catalog builder; background jobs; catalog upserts on start

Endpoints

ROUTE	PURPOSE
GET / · /about	The front page (`index.html` + `front.js`); the owner's letter
GET /api/catalog	Filtered by `category`, `year_from`/`year_to`, `country`, `group`; sorted by acclaim, newest, oldest or seeded shuffle; paged
GET /api/catalog/groups · meta · /{id}	Groups sized to the filters; counts, year ranges and countries for the controls; one item
POST /api/search	Prompt + filters → every work judged a 75%+ fit, best first. 30 per visitor per hour; 4 run at once server-wide, up to 12 more wait ≤30 s, then 503 "busy"; injection phrasing refused (400)
Other	`/robots.txt`, `/sitemap.xml`, `/favicon.ico`, `/health`, `/manifest.webmanifest`, `/sw.js`

Data

`content_items` holds the catalog (category, title, creator, year, source, url, image, synopsis, tags, subjects, tones, countries, tier, runtime, publish date); `art_images` is parked. Migration 0006 dropped all reader data; the old code is at git tag `personal-recommender-final`. Migrations run on start so the database is never deleted for a schema change.

Background jobs (APScheduler, in-process)

JOB	EVERY	DOES
label_new_items	1 hour	Claude labels up to 200 unlabeled items and up to 600 with no countries
refresh_substack	6 hours	New free posts from chosen newsletters
refresh_trending_books	1 week	Penguin Random House's ten trending books not yet in the catalog (no children's books); also shortly after each start
fetch_wanted_images	1 min	Covers/synopses for up to 15 items just shown without one
backfill_images	30 min	20 more, most acclaimed first

06 Content: catalogs and enrichment

CATEGORY	≈ ITEMS	SOURCE
Movies	1,900	Cross-referenced "greatest films" lists (1940–2021), plus 2022+ films from Cannes, Sundance, Toronto and Hong Kong
Books	3,500 + weekly	"Greatest novels" lists (1810 on); every NYT fiction and nonfiction #1 since 1976 (~1,580, from Wikipedia); PRH trending weekly; this week's NYT #1s
Albums	1,800	Rolling Stone 500 (sorts first), BBC Music Magazine's 31 classical, and ~1,300 from Latin America, South Korea and Africa via juried awards and critics' lists. Tiered instrumental or lyrical
Essays	2,000 + live	~1,500 from Longform.org picks and National Magazine Award winners; 60 public-domain Atlantic essays (pre-1931) linked to free full text; plus RSS and Substack (free posts)
Art	live	National Gallery of Art, the Met, V&A, Wikimedia Commons (parked)

The big lists are built offline (`azine-build-catalog`) into `data/*_list.py`; acclaim is how many source lists name a work, scaled within its category.

Enrichment

`uv run azine-fetch-details` fetches covers, synopses, links and runtimes for the whole catalog into `data/details/*.json`, committed and applied on every start, so a deploy ships them. It paces itself inside free quotas (OMDb 1,000/day, Google Books per key, iTunes ~20/min) and can be rerun. Anything still missing is fetched by the server's background jobs.

- **Movies:** OMDb poster, IMDb link, runtime, full plot. Accepted only if the director matches (OMDb returns namesakes).
- **Albums:** iTunes cover and Apple Music link, only from the same artist.
- **Books:** synopsis from Google Books, else Open Library; cleaned and cut near 900 characters. No covers: books and essays show their words.

Labels

Every item has **subjects** (what it's about) and **tones** (how it feels) from one shared vocabulary (`data/labels_vocab.py`), so "melancholy" means the same on a film and an album. Movies, albums and books also get up to three **countries** (today's names). Curated labels live in `data/catalog_labels.json`; Claude labels new items hourly.

07 Finding things: filters, groups and search

Filters (no Claude): any categories, a year range and countries. Undated works drop out once years are set; essays once a country is.

Embeddings: each item is a TF-IDF vector, 70% labels and 30% title and synopsis. Labels carry the signal: 91–94% of same-genre pairs land closer than the nearest 10% of random pairs (~50% with text alone).

Groups (no Claude): k-means per category (~one group per 50 items, 6–40), each named by the labels its members share ("eerie · mental health · dark"). Only labeled items are grouped.

Prompt search

1. **Check** (plain code). `prompt_guard.py` refuses injection phrasing, disguised spellings included, with a 400 and a reason. Only command forms count: "a soldier who ignores his orders" still searches.
2. **Plan** (one Sonnet call). Claude reads the prompt, not the catalog, and returns 1–4 facets, each with narrowed categories, years and countries, labels, keywords, typical works and groups. "The sea, and also funny 90s albums" is two facets.
3. **Order** (local). Each facet is a point in embedding space; works in the filters are lined up nearest first, named works at the front, facets taking turns (after Pinterest's PinnerSage). Capped at 2,000.
4. **Judge** (Haiku, batches of 60, 32 at once). Each work is scored 0–100; it fits only if every essential part of the prompt holds. Scores of 63–74 get a second look. Everything 75+ returns, best first.

WHY JUDGE EVERY WORK, AGAINST A 75% BAR

Owner's rule: return what really fits, not a fixed number. For "a girly film about finding yourself", *Roman Holiday* fits and *Pride and Prejudice* doesn't. Similarity can't make that call, and ranks *Roman Holiday* ~350th, so every work is judged. Haiku makes that affordable (a few cents) and judged better than Sonnet here; Sonnet writes far better plans. Plans are cached per prompt and filters. TF-IDF beats a neural embedding model at this size without a multi-gigabyte dependency.

08 Front end: the four letters

One template (`templates/index.html`) plus `static/js/front.js`, which builds everything after the first screen from the API. No framework: the page has four states (letters, catalog, waiting, results), and plain templates and CSS variables keep every visual choice in one hand-tunable file.

1. **Letters.** E, M, A, B down the left in Helvetica on `#FFF8F0`; "better for your brain" in the corner. Beside them, one work of art from `data/home_art.py`, the same for everyone, changing every 2 hours.

2 **Title card.** A clicked letter grows into its title card (**E**SSAYS) with the catalog's year range, holds 3 seconds (a click skips), then opens that catalog. The URL keeps state (`#movie+album`).

3 **Filter column.** The letters again, a vertical timeline, and "Vibes", the prompt search, with a sample prompt for the open category.

4 **Catalog.** 5 columns, most acclaimed first, 60 at a time. Movies and albums show covers (details on hover); essays and books show their words.

5 **Searching** (~15–25 s). Public-domain Met art from `data/landing_wall.py`, one work at a time with its details, instead of a spinner; then the results.

Styles: section 9 of `style.css` and its FRONT PAGE `:root` settings. Phones get smaller letters and 2 columns. The owner's handwriting font, "Azine Hand" (`static/fonts/azine-hand.woff2`, built from vector PDFs by `scripts/make_handwriting_font.py`), is used for `class="hand"` text such as the About heading.

09 Operations

TOPIC	TODAY
Cost	Render ~$7/month + disk; azine.io yearly; Claude ~1–5 cents per search, plus labeling
Memory	~310 MB of 512 MB (measured before the catalog; re-measure). Upgrade to 1 CPU / 2 GB near ~450 MB
Quotas	OMDb 1,000/day; Google Books per key; searches 30 per visitor per hour, 4 at once server-wide
Backups	Daily Render disk snapshots, kept ≥7 days; own copy via SQLite's backup, then `scp -s` (DEPLOY.md; the database is in WAL mode)
Logs	Render → Logs
Deploy time	A few minutes to build; ~1 minute cold start on an empty database

10 Decision log

Newest first. Entries before Sept 30, 2026 are reconstructed from the code. After any significant change, add a row and rebuild (`docs/architecture/README.md`).

WHEN	DECISION	WHY
Oct 31	**Art beside the letters, one work every 2 hours, timed by the clock; sample prompts in a larger, rounded Vibes box**	Owner's request. The clock needs no server state and shows everyone the same work. Only public-domain art: Shiraga's *Untitled* (1964) was left out because he died in 2008. Works are stored with the site, like the landing wall.
Oct 30	**Ready for many visitors at once: 4 searches at a time, uncapped database connections, SQLite in WAL mode, one shared Claude client; the catalog page pages through a cached, presorted index**	A load test showed ~15 simultaneous searches used up all 15 database connections (each held for the 15–25 s search) and every other page failed. Capping searches also keeps their ~35 Claude calls each under Anthropic's rate limits; WAL stops background writes from locking readers out. The catalog page had loaded and sorted all ~7,800 items per request; now ~11 ms instead of ~140 ms.
Oct 29	**These documents made more concise**	Owner's request.
Oct 29	**Every NYT #1 since 1976, and PRH's trending books weekly**	More books, nonfiction too. Wikipedia lists 50 years of #1s (the NYT API starts in 2008). Only #1s, so searches stay quick.
Oct 29	**Novels become Books: N is now B**	The catalog takes nonfiction too. Migration 0008 renames existing rows.
Oct 29	**No iOS app; its code deleted**	The home-screen app covers phones, and Apple rejects website wrappers.
Oct 29	**These documents restyled to be minimal**	Easier to read.
Oct 29	**Search prompts checked for prompt injection**	Anyone can type into Vibes. Plain code can't be talked round; refusals count toward the limit. A model screener was rejected (injectable, and costs per search).
Oct 28	**Public-domain Atlantic essays**	The Atlantic and The New Yorker meter articles; pre-1931 work is free.
Oct 28	**Covers and synopses fetched in bulk and committed**	The slow trickle left results bare. Movies must match the director (OMDb returned namesakes).
Oct 28	**Country labels and a country filter**	Owner's request. Claude labeled in one pass for a few dollars, instead of matching to Wikidata.

Oct 28	**Albums from Latin America, South Korea and Africa**	Albums were almost all Anglo-American; juried awards fill in.
Oct 28	**Front page: the letters down the left, a title card on click**	Owner's Canva design, after a first version with letters in the corners.
Oct 28	**Essays from Longform and the National Magazine Awards**	Essays were thinnest and had no acclaim signal; both sources are human-chosen.
Oct 27	**Search judges every work; Sonnet plans, Haiku judges**	Roman Holiday ranked too low by similarity to be judged. Haiku judges at ~1/15 the cost.
Oct 27	**azine becomes a catalog: no accounts, picks or votes**	Owner's direction. Migration 0006 drops reader data; old code at tag `personal-recommender-final`.
Sept 29 – Oct 9	**The personal board (retired Oct 27)**	Taste answer read by Claude, votes, saved items, the handwritten landing note and board labels (the font remains).
Oct 1	**Domain azine.io on Cloudflare**	azine.com is taken.
Oct 1	**Host on Render (replacing Fly.io)**	A dashboard and GitHub deploys; no-disk hosts ruled out.
Oct 1	**Refuse to start without the /data disk**	Otherwise every deploy loses the database.
Oct 1	**Stream the NGA art CSVs**	Peak memory 630 → 310 MB.
Oct 1	**Versioned stylesheet + network-first service worker**	A cached old stylesheet broke a redesign.
Oct 1	**Book synopses from Google Books / Open Library**	90% had none; Claude-written synopses ruled out.
Sept 30	**Installable web app; Docker; configurable database path**	One codebase; portable; the database survives deploys.
Sept 29	**Shared subject/tone labels**	One vocabulary reads across categories.
Sept 28	**SQLite + Alembic migrations on start**	Never delete the database to change the schema.

Earlier	**TF-IDF with PinnerSage-style clusters**	Keeps distinct tastes instead of averaging them.
Earlier	**Server-rendered pages, vanilla JS, CSS variables**	Hand-tunable, nothing to build.